🎓 MAJOR PROJECT

Serverless URL Shortener

Built using AWS Lambda · DynamoDB · API Gateway · Docker · Terraform

AWS Lambda | DynamoDB | API Gateway | Docker

Terraform | ECR | Python | WSL

Cloud Computing | Serverless Architecture | Infrastructure as Code

⚙️ Implementation

01 📁 Project Setup

Created serverless-url-shortener/ folder with 3 sub-folders: lambda/, docker/, terraform/ and root files: README.md, deploy.sh, destroy.sh

02 ⚡ Lambda Functions

Wrote create_url.py (generates shortcode + saves to DynamoDB) and redirect_url.py (looks up shortcode + redirects browser). Runtime: Python 3.12

03 🐳 Docker + ECR

Built Docker image from AWS Lambda Python 3.11 base image. Pushed to ECR repo url-shortener-repo in ap-south-1 for Lambda to pull.

04 🏗️ Terraform IaC

Wrote main.tf, variables.tf, outputs.tf to provision ALL AWS resources automatically. One terraform apply command = 8 resources created!

All AWS resources provisioned automatically — no manual console clicks needed! 🚀

🔄 Project Workflow

1. 📁 Create Folder Structure

mkdir serverless-url-shortener → mkdir lambda, docker, terraform → New-Item README.md, deploy.sh, destroy.sh

2. 📝 Write Lambda Code

create_url.py → shortcode + DynamoDB store. redirect_url.py → lookup + 301 redirect to original URL.

3. 🌐 Setup API Gateway

REST API: POST /shorten → create Lambda. GET /{shortCode} → redirect Lambda. Deploy to prod stage.

4. 🐳 Build & Push Docker

docker build → docker tag → aws ecr get-login-password → docker push to ECR in ap-south-1.

5. 🏗️ Terraform Apply

terraform -chdir=terraform apply → 8 resources: Lambda, DynamoDB, API Gateway, ECR, IAM.

6. ✅ Test the API

curl POST /shorten → get short URL (pbkYV6). Paste in browser → redirects to original URL!

Full flow: Code → Docker Image → ECR → Lambda → API Gateway → DynamoDB → Short URL in browser 🎉

What We Built & Learned

Serverless Computing

Lambda runs code on demand — no servers to manage, pay only when code runs!

Docker Containers

Packaged Lambda + dependencies in Docker. Same environment everywhere!

Infrastructure as Code

Terraform created 8 AWS resources with one command. Cloud setup = code!

REST API Design

POST /shorten creates, GET /{code} redirects. Real REST API on AWS!

NoSQL Database

DynamoDB stores URL mappings with millisecond reads — scales to millions!

Full DevOps Cycle

Code → Docker → ECR → Lambda → API Gateway → Test → Destroy. Complete!

Stack: Python · AWS Lambda · DynamoDB · API Gateway · Docker · ECR · Terraform · WSL | Region: ap-south-1

Step 1 — Create Root Project Folder

```
PS C:\Users\JAY> mkdir serverless-url-shortener


    Directory: C:\Users\JAY

Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----         21-04-2026     15:05                serverless-url-shortener
```

The command 'mkdir serverless-url-shortener' is run in PowerShell to create the main project folder at C:\Users\JAY. This is the first step of any organized project — keeping all files in one dedicated directory makes it easy to manage and share.

mkdir creates a new directory. The folder serverless-url-shortener will hold all sub-folders and files for the entire project — Lambda code, Docker config, and Terraform files all live here.

Step 2 — Create Sub-Folders: lambda, docker, terraform

```
PS C:\Users\JAY\serverless-url-shortener> mkdir lambda, docker, terraform


    Directory: C:\Users\JAY\serverless-url-shortener


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----        21-04-2026     15:06                lambda
d-----        21-04-2026     15:06                docker
d-----        21-04-2026     15:06                terraform
```

Three sub-folders are created inside the project root: lambda/ for Python function code, docker/ for the Dockerfile, and terraform/ for infrastructure configuration. This separation keeps each part of the project clean and independent.

mkdir lambda, docker, terraform creates all three folders at once. Separating code (lambda), packaging (docker), and infrastructure (terraform) into different folders is a standard best practice for serverless projects.

Step 3 — Create Root Files: README.md, deploy.sh, destroy.sh

```
PS C:\Users\JAY\serverless-url-shortener> New-Item -ItemType File -Path README.md, deploy.sh, destroy.sh


    Directory: C:\Users\JAY\serverless-url-shortener


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a----         21-04-2026     15:07              0 README.md
-a----         21-04-2026     15:07              0 deploy.sh
-a----         21-04-2026     15:07              0 destroy.sh
```

Three files are created at the project root: README.md documents the project, deploy.sh automates the full deployment with one command, and destroy.sh removes all AWS resources with one command. This avoids repeating many manual steps every time.

New-Item -ItemType File creates empty files in PowerShell. deploy.sh and destroy.sh are shell scripts that will later be filled with automation commands — they make the project easier to run by any team member with a single command.

Step 4 — Create Lambda Function Files

```
PS C:\Users\JAY\serverless-url-shortener> New-Item -ItemType File -Path lambda/create_url.py, lambda/redirect_url.py, lambda/requirements.txt


    Directory: C:\Users\JAY\serverless-url-shortener\lambda


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a----         21-04-2026     15:07              0 create_url.py
-a----         21-04-2026     15:07              0 redirect_url.py
-a----         21-04-2026     15:07              0 requirements.txt
```

Inside the lambda/ folder, three files are created: create_url.py handles POST requests — it generates a random shortcode and stores the URL mapping in DynamoDB. redirect_url.py handles GET requests — it looks up the shortcode and redirects the browser to the original URL. requirements.txt lists all Python dependencies needed.

Two separate Lambda files follow the Single Responsibility Principle — each function does one job. requirements.txt tells Docker which Python packages to install when building the container image.

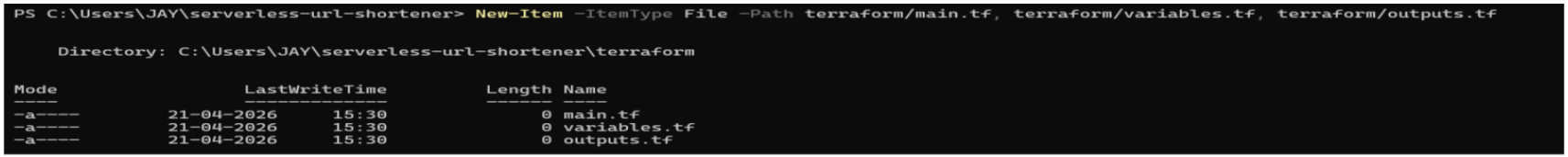
Step 5 — Create Docker Files: Dockerfile and .dockerignore

```
PS C:\Users\JAY\serverless-url-shortener>  New-Item -ItemType File -Path docker/Dockerfile, docker/.dockerignore


    Directory: C:\Users\JAY\serverless-url-shortener\docker


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a----         21-04-2026     15:07              0 Dockerfile
-a----         21-04-2026     15:07              0 .dockerignore
```

Dockerfile defines how to build the container image for Lambda. It starts from AWS's official Lambda Python 3.11 base image and adds our code and dependencies on top. .dockerignore tells Docker to skip unnecessary files like .git and __pycache__ to keep the image small.

Docker is used because Lambda with custom Python dependencies is deployed as a container image. The AWS base image ensures the Python environment inside Docker exactly matches what Lambda expects — same code runs the same way locally and in the cloud.

Step 6 — Create Terraform Files: main.tf, variables.tf, outputs.tf



Three Terraform files are created: main.tf defines all AWS resources to be created (Lambda, DynamoDB, API Gateway, ECR, IAM roles), variables.tf holds configurable input values like region and account ID, and outputs.tf prints important values like API URLs and Lambda ARNs after deployment.

Infrastructure as Code means the entire AWS setup is written in files and version-controlled just like application code. With these three files, anyone can recreate the exact same infrastructure in any AWS account using just one command.

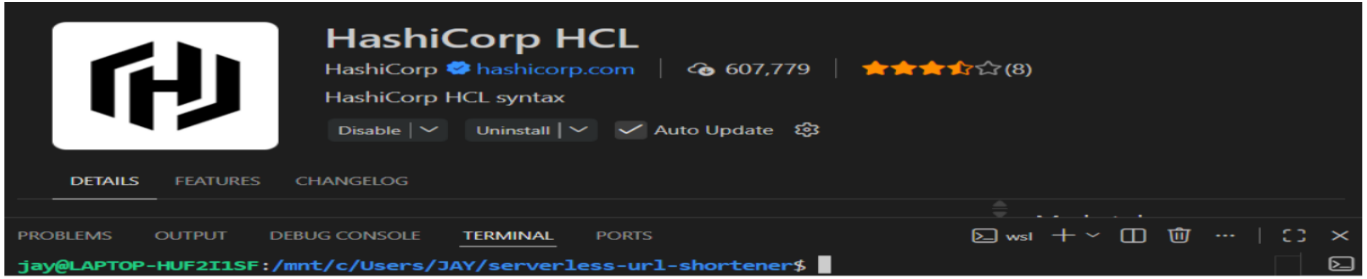
Step 7 — Verify Terraform Installation

```
jay@LAPTOP-HUF2I1SF:/mnt/c/Users/JAY/serverless-url-shortener$ terraform --version
Terraform v1.14.9
on linux_amd64
```

Running 'terraform --version' in the WSL terminal confirms Terraform v1.14.9 is installed on linux_amd64. This is run inside WSL (Windows Subsystem for Linux) which provides a Linux environment on Windows for using AWS and Terraform tools.

Always verify tool versions before starting a project. Different versions of Terraform may have different syntax or provider compatibility. v1.14.9 is compatible with the hashicorp/aws provider v5.100.0 used in this project.

Step 8 — VS Code with HashiCorp HCL Extension



The HashiCorp HCL extension is installed in VS Code to provide syntax highlighting, auto-complete, and error checking for .tf Terraform files. The integrated WSL terminal at the bottom lets us edit code and run commands in the same window without switching applications.

The HCL extension catches syntax mistakes in Terraform files before running terraform apply — this is important because errors in infrastructure code can create wrong AWS resources. The VS Code + WSL setup is the standard development environment for cloud projects on Windows.

Step 9 — Verify Docker Installation

```
jay@LAPTOP-HUF2I1SF:/mnt/c/Users/JAY/serverless-url-shortener$ docker --version
Docker version 28.3.0, build 38b7060
```

Running 'docker --version' in WSL confirms Docker version 28.3.0 is installed and working. Docker is required to build the container image that packages the Lambda functions along with all their Python dependencies before pushing to ECR.

Lambda with custom dependencies must be deployed as a container image rather than a simple zip file when packages are too large or complex. Docker 28.3.0 supports multi-platform builds needed for the AWS Lambda linux/amd64 runtime.

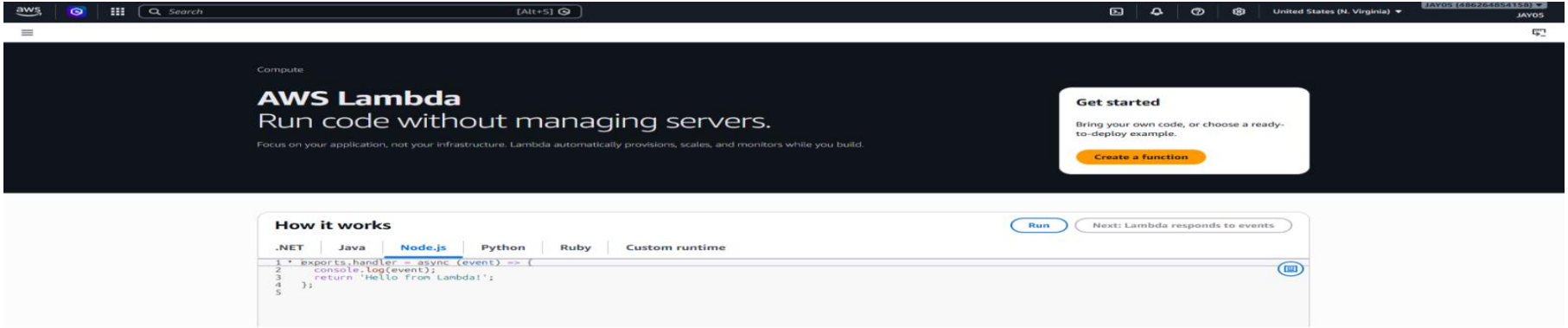
Step 10 — Verify boto3 (AWS SDK for Python)

```
jay@LAPTOP-HUF2I1SF:/mnt/c/Users/JAY/serverless-url-shortener$ python3 -m pip show boto3
Name: boto3
Version: 1.34.46
Summary: The AWS SDK for Python
Home-page: https://github.com/boto/boto3
Author: Amazon Web Services
Author-email:
License: Apache License 2.0
Location: /usr/lib/python3/dist-packages
Requires:
Required-by:
```

Running 'python3 -m pip show boto3' confirms boto3 version 1.34.46 is installed. boto3 is the official AWS SDK for Python and is used inside the Lambda functions to interact with DynamoDB — storing short URL mappings in create_url.py and reading them in redirect_url.py.

boto3 must be listed in requirements.txt so it gets installed inside the Docker image during build. Without boto3, the Lambda functions cannot communicate with DynamoDB or any other AWS service — it is the bridge between Python code and AWS cloud services.

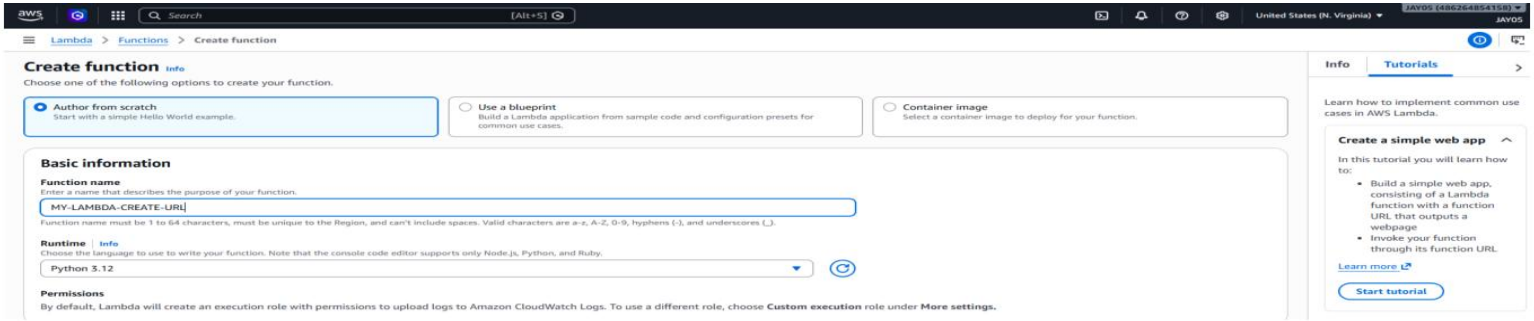
Step 11 — AWS Lambda Console Overview



The AWS Lambda console is opened showing the Lambda service landing page. Lambda is the core compute service of this project — it runs the Python functions without any server management. Lambda automatically provisions resources, scales based on incoming traffic, and charges only for actual execution time.

Lambda is chosen for the URL shortener because traffic is unpredictable — some short links may get thousands of clicks while others are rarely used. Lambda scales from zero to thousands of concurrent executions automatically, and the AWS Free Tier includes 1 million Lambda requests per month.

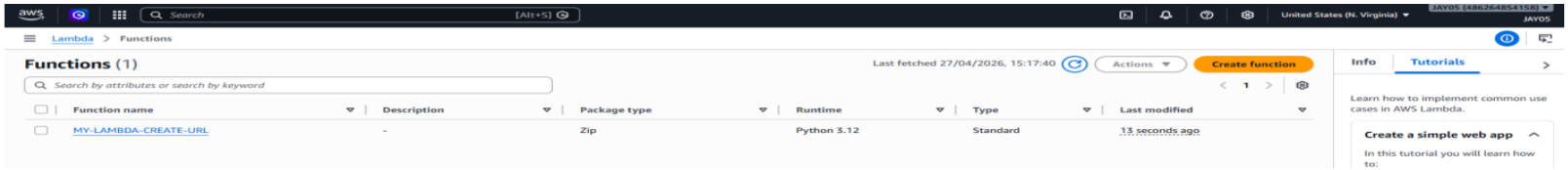
Step 12 — Create Lambda Function: MY-LAMBDA-CREATE-URL



The first Lambda function is created from scratch with the name MY-LAMBDA-CREATE-URL, Python 3.12 runtime, in the us-east-1 (N. Virginia) region. This function will handle POST /shorten API requests — it generates a 6-character shortcode and saves the URL mapping to DynamoDB.

The function is created in the console first for manual testing and understanding. Later, Terraform will recreate the final production version in ap-south-1 using the Docker container image. Python 3.12 is the latest stable runtime with best performance and security patches.

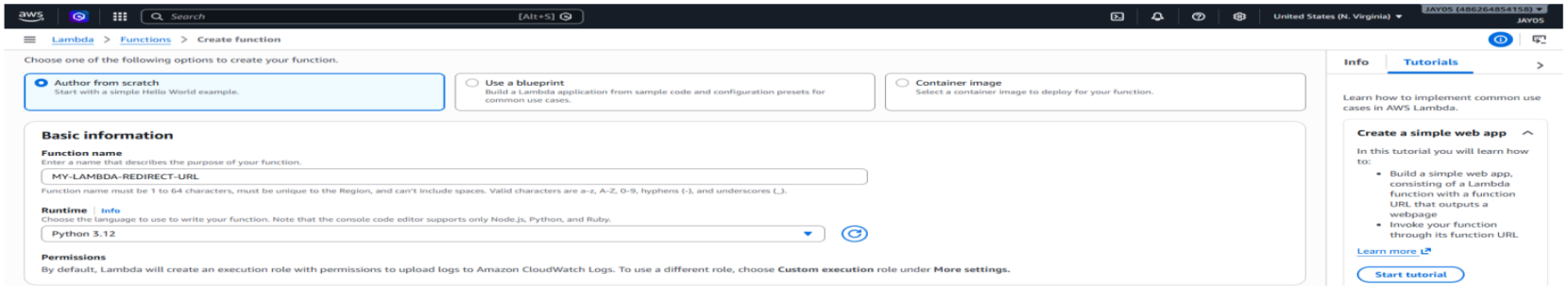
Step 13 — Lambda Function List: CREATE-URL Confirmed



The Lambda Functions list now shows MY-LAMBDA-CREATE-URL was created 13 seconds ago with Zip package type and Python 3.12 runtime. Seeing it appear in the list confirms AWS successfully provisioned the function and it is ready to have code uploaded to it.

Package type 'Zip' means code is uploaded as a compressed zip file for this console-created version. The final Terraform-deployed version uses 'Image' type meaning it runs from the Docker container stored in ECR. Both methods work — Zip is used first for quick testing.

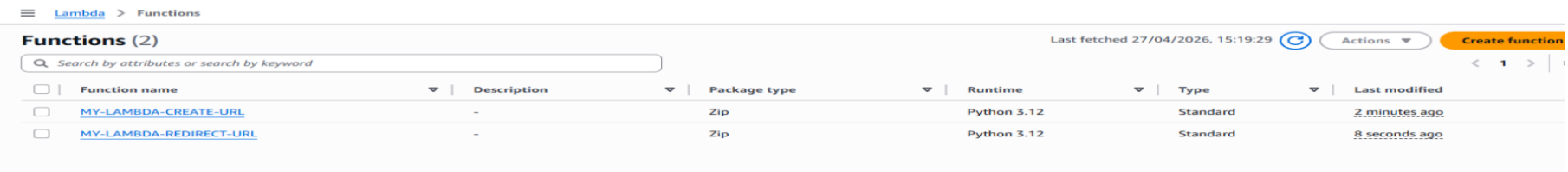
Step 14 — Create Lambda Function: MY-LAMBDA-REDIRECT-URL



The second Lambda function MY-LAMBDA-REDIRECT-URL is created with the same Python 3.12 runtime. This function handles GET /{shortCode} requests — it receives the short code from the URL, looks it up in DynamoDB, and returns a 301 redirect response that tells the browser to navigate to the original long URL.

A 301 redirect means 'Moved Permanently' — the browser follows this automatically and navigates to the Location header value without any user action. Having two separate functions (one create, one redirect) means each can be updated, scaled, and monitored independently.

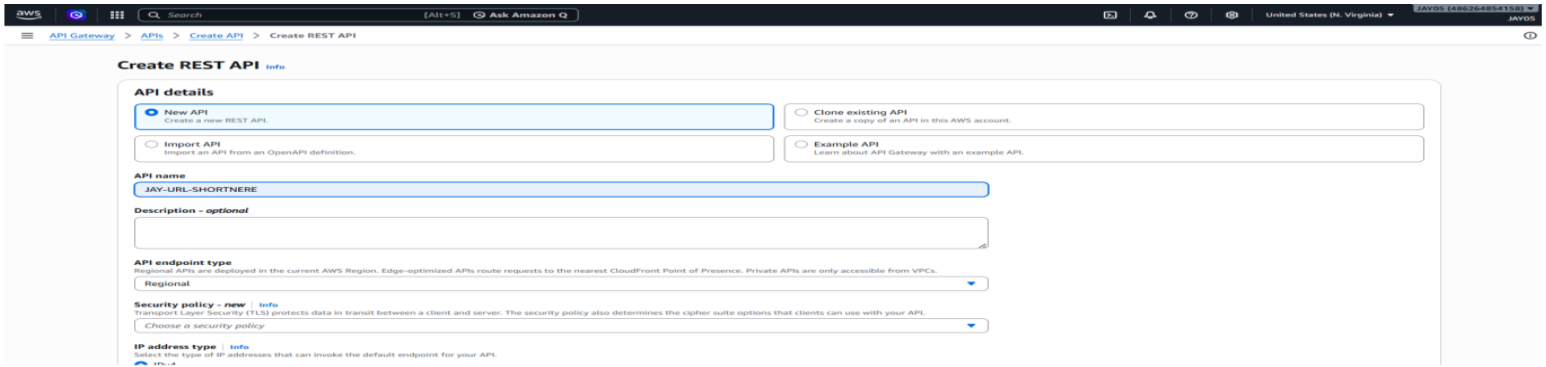
Step 15 — Both Lambda Functions Listed



The Lambda console now shows both functions: MY-LAMBDA-CREATE-URL (created 2 minutes ago) and MY-LAMBDA-REDIRECT-URL (created 8 seconds ago). Both use Python 3.12 runtime, Zip package type, and Standard function type. The serverless compute layer of the URL shortener is now in place.

Confirming both functions appear in the list with correct runtime and type verifies they are both active and ready. In the final Terraform deployment, these are replaced by container image-based functions in the ap-south-1 region. This console phase was for learning and manual testing.

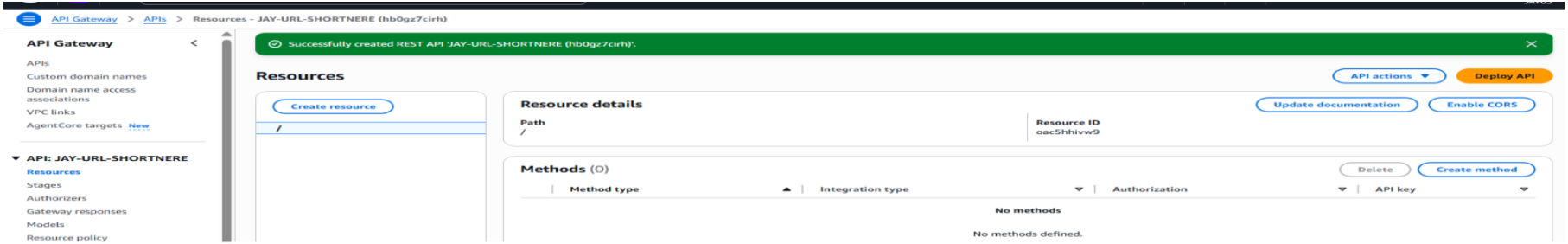
Step 16 — Create REST API in API Gateway



A new REST API named JAY-URL-SHORTNERE is created in API Gateway with Regional endpoint type. API Gateway acts as the public HTTP entry point for the URL shortener — it receives incoming requests and routes them to the correct Lambda function based on the path and HTTP method.

Regional endpoint type means the API is deployed within the current AWS region (ap-south-1 for India) rather than distributed globally via CloudFront. This is sufficient for a project targeting regional users and avoids additional CloudFront configuration complexity.

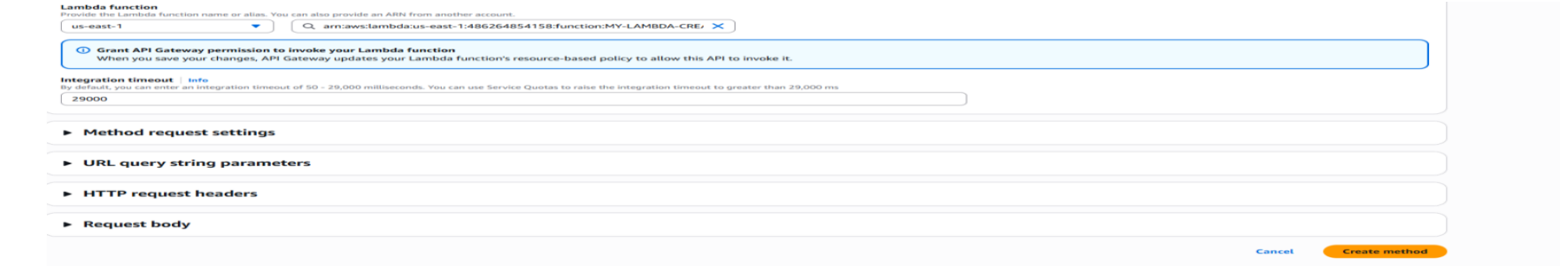
Step 17 — REST API Created Successfully



The API Gateway console shows the success message: 'Successfully created REST API JAY-URL-SHORTNERE (hb0gz7cirh)'. The Resources panel shows the root path '/' with Resource ID oac5hhivw9. No methods are defined yet — the next steps are adding the /shorten and /{shortCode} resources with Lambda integrations.

The API ID hb0gz7cirh appears in the final Invoke URL and all API references. The root '/' resource is the starting point — all API paths like /shorten and /{shortCode} are created as child resources under this root, following standard REST API structure.

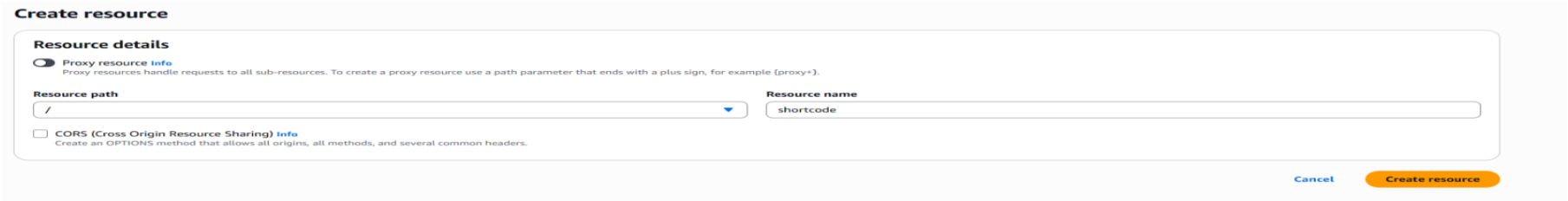
Step 18 — Link API Gateway POST Method to Create Lambda



The POST method on the root resource is configured to integrate with the MY-LAMBDA-CREATE-URL function using its full ARN. Lambda proxy integration is enabled — this passes the complete HTTP request including body, headers, and query parameters directly to the Lambda function as a structured JSON event.

Lambda proxy integration is the simplest and most powerful integration option. The Lambda function receives the full request and has complete control over the response — it sets the status code, headers, and body. Integration timeout is set to 29 seconds, the maximum allowed.

Step 19 — Create Resource '/shortcode'



A new API resource named 'shortcode' is created under the root path, creating the URL path /shortcode. In the final Terraform configuration this is set as /{shortCode} with a path parameter — allowing any value like /pbkYV6 to be captured and passed to the redirect Lambda function.

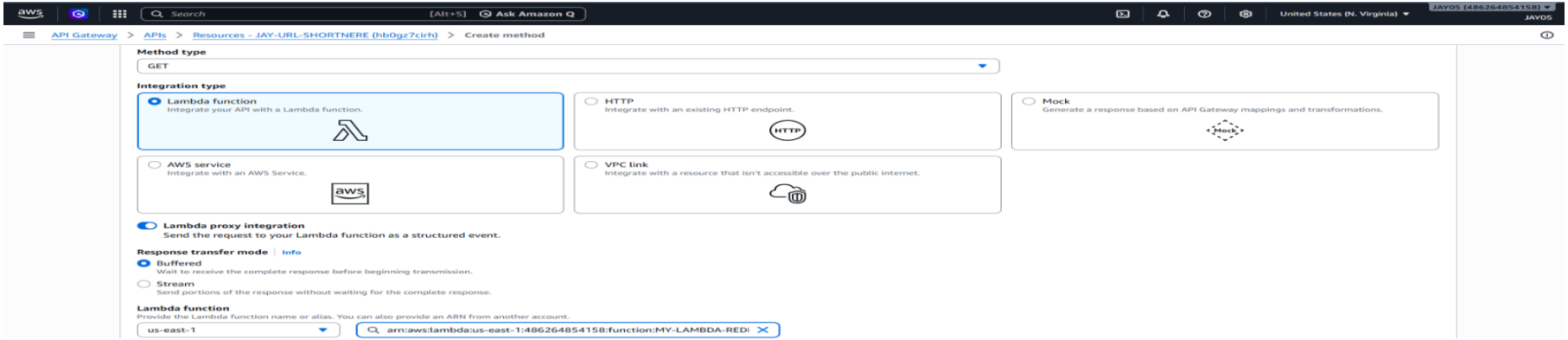
Path parameters in API Gateway use curly braces like {shortCode} to capture variable parts of the URL. When someone visits the API with /prod/pbkYV6, API Gateway extracts 'pbkYV6' from the path and includes it in the event passed to the Lambda function for DynamoDB lookup.

Step 20 — Resources Panel Shows POST and /shortcode

The API Resources panel now shows the structure taking shape: the root '/' has a POST method connected to the create Lambda, and the '/shortcode' resource has been created with Resource ID lsowqv. The next step is adding a GET method to /shortcode linked to the redirect Lambda.

This RESTful API design follows standard HTTP conventions: POST is used to create a new resource (the short URL), and GET is used to retrieve or follow a resource (redirect to the original URL). This makes the API predictable and easy to understand for any developer.

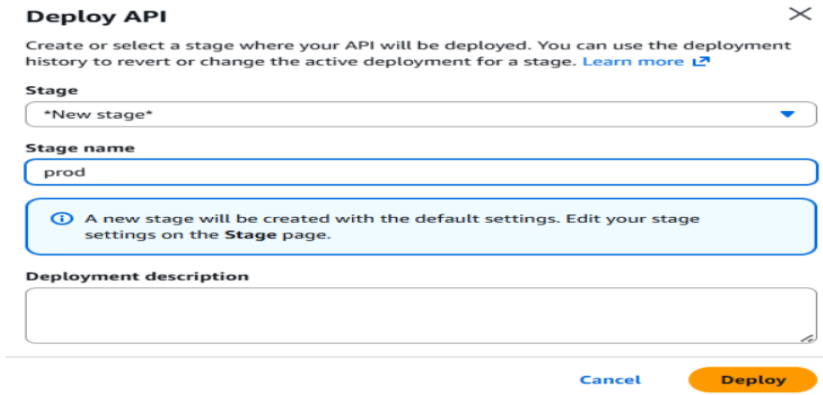
Step 21 — GET Method on /shortcode Linked to Redirect Lambda



A GET method is created on the /shortcode resource and linked to the MY-LAMBDA-REDIRECT-URL function using its ARN. Lambda proxy integration is enabled with buffered response mode. When a browser visits the short URL, this GET request flows through API Gateway to the redirect Lambda which performs the DynamoDB lookup.

GET is the correct HTTP method for URL redirection because browsers automatically send GET requests when visiting a URL and automatically follow 301 redirect responses. The Lambda function returns a response with statusCode 301 and a Location header pointing to the original URL.

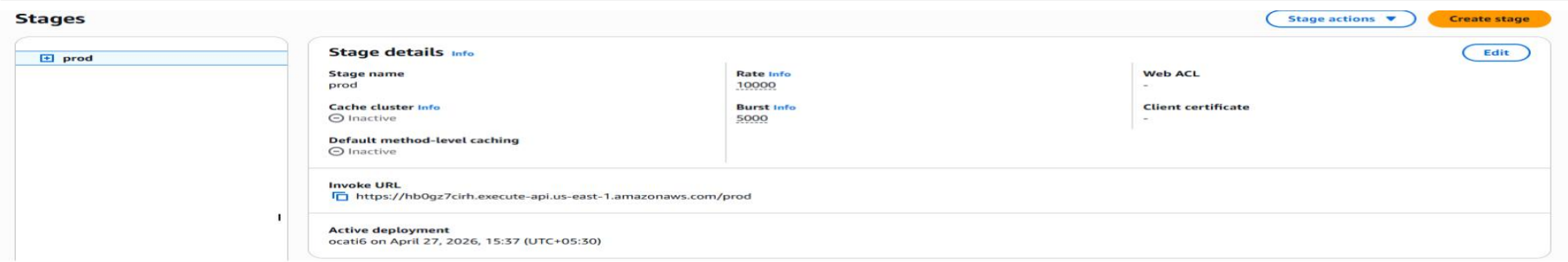
Step 22 — Deploy API to 'prod' Stage



The API is deployed to a new stage named 'prod'. Deploying creates a snapshot of all the API configuration and makes the API publicly accessible via an HTTPS Invoke URL. Before deployment, the API only exists as configuration in the console and cannot receive any real traffic.

A stage in API Gateway is like a deployment environment. The name 'prod' indicates production. The stage name appears in the Invoke URL as part of the path. Multiple stages like 'dev', 'staging', and 'prod' can coexist with different Lambda function versions or settings.

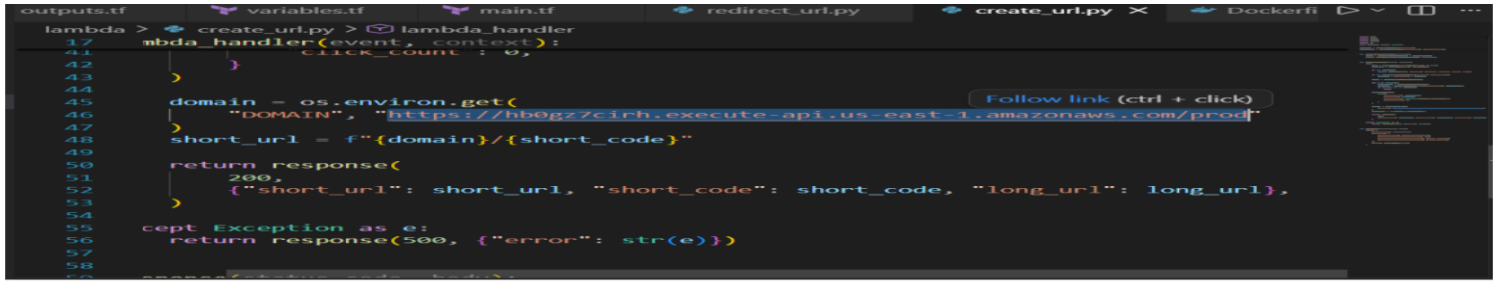
Step 23 — Prod Stage Live with Invoke URL



The prod stage details show the Invoke URL: https://hb0gz7cirh.execute-api.us-east-1.amazonaws.com/prod. Rate limit is 10,000 requests per second with a burst limit of 5,000. This URL is used as the base domain in the create_url Lambda to build the final short URL returned to the user.

The Invoke URL is the public HTTPS address of the API. It is stored as a DOMAIN environment variable in the Lambda function so short URLs are constructed as {DOMAIN}/{shortCode}. Rate limits protect the API from being overwhelmed and incurring unexpected AWS costs.

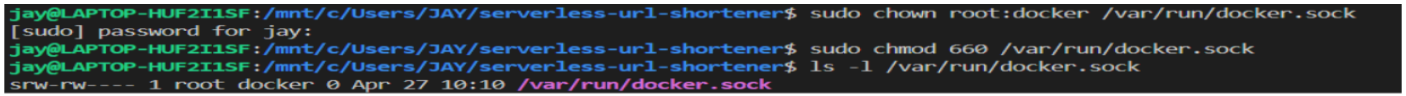
Step 24 — Lambda Code Using the API Domain



In create_url.py the DOMAIN environment variable is read using os.environ.get('DOMAIN', default_url). The short URL is built using Python f-string formatting as f'{domain}/{short_code}'. The function returns HTTP 200 with a JSON body containing short_url, short_code, and long_url.

Using an environment variable for the domain makes the code portable — the same Lambda function works with different API Gateway URLs just by changing the environment variable value. No code changes are needed when moving from the us-east-1 testing setup to the ap-south-1 Terraform production deployment.

Step 25 — Fix Docker Socket Permissions in WSL



In WSL, the Docker socket at /var/run/docker.sock needs permission fixes before non-root users can run Docker commands. The commands sudo chown root:docker and sudo chmod 660 are run to give the docker group read-write access. The ls -l output confirms the correct permissions are now set.

This is a common setup issue in WSL. Without these permission changes, running docker build or docker push gives a 'permission denied while trying to connect to Docker daemon socket' error. chmod 660 means the root user and docker group can read and write — regular users without docker group membership cannot.

Step 26 — Verify AWS CLI Identity

```
jay@LAPTOP-HUF2I1SF:/mnt/c/Users/JAY/serverless-url-shortener$ aws sts get-caller-identity
{
    "UserId": "AIDAXCN5E22HI6KWMVBBO",
    "Account": "486264854158",
    "Arn": "arn:aws:iam::486264854158:user/JAY@012"
}
```

Running 'aws sts get-caller-identity' confirms the AWS CLI is authenticated and shows the active identity: UserId AIDAXCN5E22HI6KWMVBB0, Account 486264854158, and ARN arn:aws:iam::486264854158:user/JAY@012. This verifies the correct AWS account and IAM user are active before running any deployment commands.

Always run this command before deploying to AWS to confirm you are using the correct account. Account ID 486264854158 appears in ECR repository URIs, Lambda ARNs, and IAM role ARNs throughout this project. Deploying to the wrong account by mistake could result in unexpected charges.

Step 27 — Create ECR Repository

```
jay@LAPTOP-HUF2I1SF:/mnt/c/Users/JAY/serverless-url-shortener$ aws ecr create-repository --repository-name url-shortener-repo --region ap-so
uth-1
{
    "repository": {
        "repositoryArn": "arn:aws:ecr:ap-south-1:486264854158:repository/url-shortener-repo",
        "registryId": "486264854158",
        "repositoryName": "url-shortener-repo",
        "repositoryUri": "486264854158.dkr.ecr.ap-south-1.amazonaws.com/url-shortener-repo",
        "createdAt": "2026-05-01T10:55:50.232000+00:00",
        "imageTagMutability": "MUTABLE",
        "imageScanningConfiguration": {
            "scanOnPush": false
        },
        "encryptionConfiguration": {
            "encryptionType": "AES256"
        }
```

An ECR (Elastic Container Registry) repository named url-shortener-repo is created in ap-south-1 using the AWS CLI. The response shows the repositoryUri: 486264854158.dkr.ecr.ap-south-1.amazonaws.com/url-shortener-repo. This URI is where the Docker image will be pushed and where Lambda will pull it from.

ECR is AWS's managed private Docker registry. Storing the Lambda image in ECR within the same AWS account and region as Lambda means the image can be pulled securely without going through the public internet. The repository is configured with AES256 encryption and MUTABLE image tags.

Step 28 — Docker Login to ECR

```
jay@LAPTOP-HUF2I1SF:/mnt/c/Users/JAY/serverless-url-shortener$ aws ecr get-login-password --region us-east-1 | \
docker login --username AWS --password-stdin 486264854158.dkr.ecr.us-east-1.amazonaws.com
Login Succeeded
```

The command generates a temporary authentication token from AWS and pipes it directly to docker login for the ECR registry endpoint. The 'Login Succeeded' message confirms Docker is now authenticated and can push and pull images from the ECR repository for the next 12 hours.

ECR uses short-lived tokens rather than permanent passwords for security. The aws ecr get-login-password command generates a 12-hour token and the pipe symbol passes it directly to docker login. Without this step, docker push commands fail with an authentication error.

Step 29 — Build Docker Image

```
jay@LAPTOP-HUF2I1SF:/mnt/c/Users/JAY/serverless-url-shortener$  docker build -f docker/Dockerfile -t url-shortener-repo .
[+] Building 33.2s (10/10) FINISHED                                                                                 docker:default
 => [internal] load build definition from Dockerfile                                                                         0.1s
 => => transferring dockerfile: 316B                                                                                         0.1s
 => [internal] load metadata for public.ecr.aws/lambda/python:3.11                                                           4.8s
 => [internal] load .dockerignore                                                                                            0.1s
 => => transferring context: 2B                                                                                              0.1s
 => [1/5] FROM public.ecr.aws/lambda/python:3.11@sha256:2a18e270d1a4cdcf56930bb007e64044f9160957c743badf07f62c0cb3e75964     25.0s
```

The command docker build -f docker/Dockerfile -t url-shortener-repo . builds the container image in 33.2 seconds completing all 10 build steps. The build pulls the AWS Lambda Python 3.11 base image from the public ECR registry, loads the .dockerignore rules, and copies the Lambda function code into the image.

The -f flag specifies the Dockerfile location, -t sets the image tag name, and the dot at the end sets the build context to the current directory. The AWS Lambda Python 3.11 base image from public.ecr.aws is the official image designed specifically to run inside AWS Lambda.

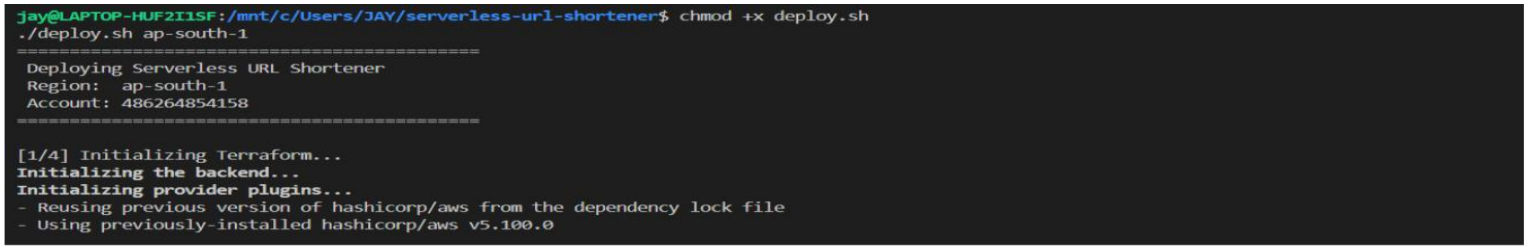
Step 30 — Tag and Push Docker Image to ECR

```
jay@LAPTOP-HUF2I1SF:/mnt/c/Users/JAY/serverless-url-shortener$ docker tag url-shortener-repo:latest 486264854158.dkr.ecr.us-east-1.amazonaws.com/u
rl-shortener-repo:latest
jay@LAPTOP-HUF2I1SF:/mnt/c/Users/JAY/serverless-url-shortener$ docker push 486264854158.dkr.ecr.us-east-1.amazonaws.com/url-shortener-repo:latest
The push refers to repository [486264854158.dkr.ecr.us-east-1.amazonaws.com/url-shortener-repo]
c55176c1a45e: Pushed
a1aaa884949b: Pushed
daf17606c1f6: Pushed
fdf5e8cffa33: Pushed
4b6d42e6844e: Pushed
72b43d245193: Pushed
626e8abb4e79: Pushed
685683a0c915: Pushed
fec0d3cb6efb: Pushed
89d695272527: Pushed
393064f630e5: Pushed
latest: digest: sha256:8157bd30c17442f7d460427112c8c5d9be749b470669a1b69254d9782ee8344b size: 856
```

The image is tagged with the full ECR URI using docker tag, then pushed to ECR using docker push. All 11 image layers are uploaded successfully. The final output shows the sha256 digest sha256:8157bd30... which uniquely identifies this exact version of the image stored in ECR.

Docker images are stored as layers — only changed layers need to be re-uploaded on subsequent pushes, making updates faster. The sha256 digest is a fingerprint of the image — Terraform references this exact URI when creating the Lambda function to ensure the deployed code exactly matches what was built.

Step 31 — Run deploy.sh — Terraform Initializes



After making the script executable with chmod +x, running ./deploy.sh ap-south-1 starts the automated deployment. The script displays the deployment target (Region: ap-south-1, Account: 486264854158) and begins Step 1 of 4 — Terraform initialization, which downloads and caches the required provider plugins.

deploy.sh automates the four deployment steps in sequence: Terraform init, Docker build, ECR push, and Terraform apply. Running one script instead of four separate commands reduces the chance of missing a step or making an error. Terraform reuses the previously downloaded hashicorp/aws v5.100.0 provider from the lock file.

Step 32 — Terraform Apply Shows Execution Plan

```
jay@LAPTOP-HUF2I1SF:/mnt/c/Users/JAY/serverless-url-shortener$ terraform -chdir=terraform apply

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
  + create

Terraform will perform the following actions:

  # aws_api_gateway_deployment.url_shortener will be created
  + resource "aws_api_gateway_deployment" "url_shortener" {
      + created_date  = (known after apply)
      + execution_arn = (known after apply)
      + id            = (known after apply)
      + invoke_url    = (known after apply)
      + rest_api_id   = (known after apply)
    }
```

terraform apply generates an execution plan showing exactly what resources will be created before making any changes. The plan shows aws_api_gateway_deployment.url_shortener will be created with attributes like created_date, execution_arn, id, invoke_url, and rest_api_id — all shown as 'known after apply' since AWS generates these values.

Terraform always shows a plan before applying changes — this is a critical safety feature. Developers can review the planned changes and confirm before Terraform touches any real AWS resources. '(known after apply)' means the value will be assigned by AWS after the resource is created and Terraform cannot predict it in advance.

Step 33 — Terraform Apply Complete: 8 Resources Created

```
Apply complete! Resources: 8 added, 0 changed, 0 destroyed.

Outputs:

api_base_url = "https://qllx65ml7b.execute-api.ap-south-1.amazonaws.com/prod"
dynamodb_table_name = "URLShortener"
ecr_repository_url = "486264854158.dkr.ecr.ap-south-1.amazonaws.com/url-shortener-repo"
lambda_create_arn = "arn:aws:lambda:ap-south-1:486264854158:function:url-shortener-create"
lambda_redirect_arn = "arn:aws:lambda:ap-south-1:486264854158:function:url-shortener-redirect"
redirect_endpoint = "https://qllx65ml7b.execute-api.ap-south-1.amazonaws.com/prod/{shortCode}"
shorten_endpoint = "https://qllx65ml7b.execute-api.ap-south-1.amazonaws.com/prod/shorten"
```

Terraform apply completes successfully: 8 resources added, 0 changed, 0 destroyed. The outputs section shows all important values: api_base_url, dynamodb_table_name (URLShortener), ecr_repository_url, both Lambda function ARNs (url-shortener-create and url-shortener-redirect), redirect_endpoint, and shorten_endpoint in ap-south-1.

The 8 resources Terraform created are: DynamoDB table, ECR repository, Lambda create function, Lambda redirect function, API Gateway REST API, API Gateway resource, API Gateway deployment, and IAM execution role. Terraform handled all creation order and dependency resolution automatically — tasks that would take 30 minutes in the console took about 2 minutes with IaC.

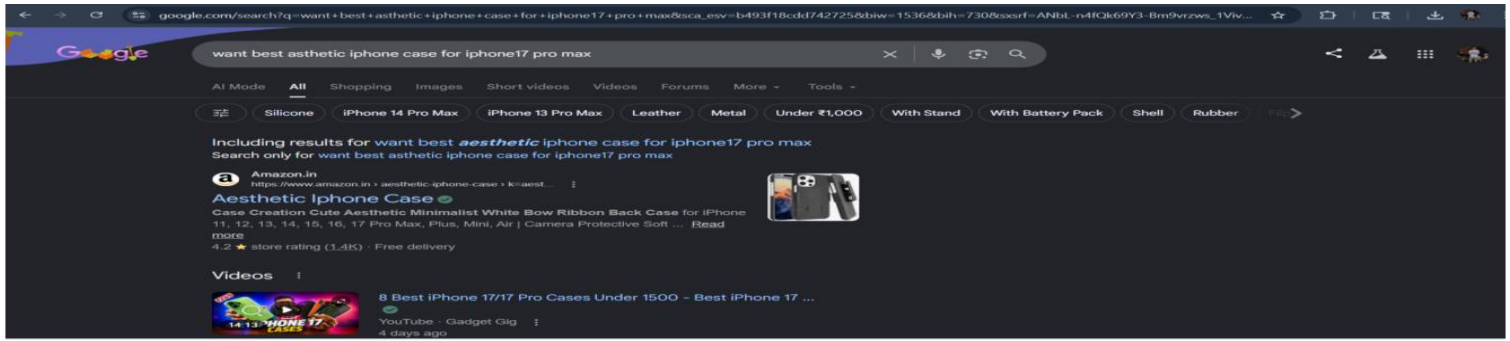
Step 34 — Terraform Output Shows All Endpoints

```
jay@LAPTOP-HUF2I1SF:/mnt/c/Users/JAY/serverless-url-shortener$ terraform -chdir=terraform output
api_base_url = "https://qllx65ml7b.execute-api.ap-south-1.amazonaws.com/prod"
dynamodb_table_name = "URLShortener"
ecr_repository_url = "486264854158.dkr.ecr.ap-south-1.amazonaws.com/url-shortener-repo"
lambda_create_arn = "arn:aws:lambda:ap-south-1:486264854158:function:url-shortener-create"
lambda_redirect_arn = "arn:aws:lambda:ap-south-1:486264854158:function:url-shortener-redirect"
redirect_endpoint = "https://qllx65ml7b.execute-api.ap-south-1.amazonaws.com/prod/{shortCode}"
shorten_endpoint = "https://qllx65ml7b.execute-api.ap-south-1.amazonaws.com/prod/shorten"
```

Running 'terraform -chdir=terraform output' prints all the configured output values: the shorten endpoint URL (https://qllx65ml7b.execute-api.ap-south-1.amazonaws.com/prod/shorten) for POST requests and the redirect endpoint URL with the {shortCode} path parameter for GET requests. These are the two live API endpoints.

Terraform outputs are declared in outputs.tf and are stored in the Terraform state file after apply. Running terraform output retrieves them at any time without logging into the AWS console. The shorten_endpoint and redirect_endpoint are the two URLs needed to use and test the URL shortener service.

Step 35 — Prepare a Long Google URL for Testing



A real Google search URL for 'want best aesthetic iPhone case for iPhone 17 pro max' is prepared as a test input. This URL contains many tracking and session parameters making it very long — well over 200 characters. This long URL will be sent to the POST /shorten endpoint to generate a short link.

Real-world URLs often contain dozens of tracking parameters appended by Google, analytics tools, and ad platforms. These make URLs impossible to share in messages or remember. The URL shortener replaces the entire long URL with a 6-character code — this test validates the system handles real URLs correctly.

Step 36 — curl POST to Shorten Endpoint

```
jay@LAPTOP-HUF2I1SF:/mnt/c/Users/JAY/serverless-url-shortener$ curl -X POST "https://qllx65ml7b.execute-api.ap-south-1.amazonaws.com/prod/shorten" \
  -H "Content-Type: application/json" \
  -d '{"url":"https://www.google.com/search?q=want+best+asthetic+iphone+case+for+iphone17+pro+max&sca_esv=b493f18cdd742725&biw=1536&bih=730&sxsrf=ANbL-n4fQk69Y3-Bm9vrzws_1VivZ-x5WQ%3A1777636165026&ei=RZP0aamoAf6C1e8PhNXS8Q4&ved=0ahUKEwjp3p_hgpiUAxV-QfUHHYSqNO4Q4dUDCBM&uact=5&oq=want+best+asthetic+iphone+case+for+iphone17+pro+max&gs_lp=Egxnd3Mtd2l6LXNlcnAiM3dhbnQgYmVzdCBhc3RoZXRpYyBpcGhvbmUgY2FzZSBmb3IgaXBob25lMTcgcHJvIG1heDIFEAAY7wUyBRAAGO8FMgUQABjvBTIFEAAY7wVIhxRQ_wFYjRJwAXgBkAEAmAHDAaAB0gqqAQMwLji4AQPIAQD4AQGYAgigAsIJwgIKEAAYRxjWBBiwA8ICBBAhGArCAggQABiJBRiiBJgDAOIDBRIBMSBAiAYBkAYIkgcDMS43oAfVKbIHAzAuN7gHuQnCBwUxLjIuNcgHGoAIAQ&sclient=gws-wiz-serp"}
```

The curl command sends a POST request to the shorten endpoint with Content-Type application/json header and the long Google URL in the JSON request body. This triggers the complete serverless pipeline: curl sends the request to API Gateway which invokes the create Lambda which generates a shortcode and stores it in DynamoDB.

curl -X POST sets the HTTP method, -H adds a header, and -d provides the request body. The request flows through HTTPS to API Gateway, is routed to MY-LAMBDA-CREATE-URL, which calls DynamoDB put_item() to store the shortcode-to-longURL mapping, then returns the short URL in the response.

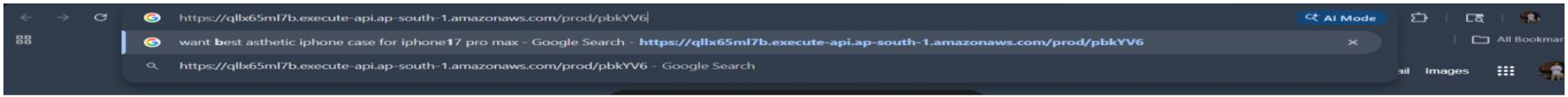
Step 37 — API Returns the Generated Short URL


```
{"short_url": "https://qllx65ml7b.execute-api.ap-south-1.amazonaws.com/prod/pbkYV6", "short_code": "pbkYV6", "long_url": "https://www.google
.com/search?q=want+best+asthetic+iphone+case+for+iphone17+pro+max&sca_esv=b493f18cdd742725&biw=1536&bih=730&sxsrf=ANbL-n4fQk69Y3-Bm9vrzws_1V
ivZ-x5WQ%3A1777636165026&ei=RZP0aamoAf6C1e8PhNXS8Q4&ved=0ahUKEwjp3p_hgpiUAxV-QfUHHYSqNO4Q4dUDCBM&uact=5&oq=want+best+asthetic+iphone+case+fo
r+iphone17+pro+max&gs_lp=Egxnd3Mtd2l6LXNlcnAiM3dhbnQgYmVzdCBhc3RoZXRpYyBpcGhvbmUgY2FzZSBmb3IgaXBob25lMTcgcHJvIG1heDIFEAAY7wUyBRAAGO8FMgUQABj
vBTIFEAAY7wVIhxRQ wFYjRJwAXgBkAEAmAHDAaAB0gqqAQMwLji4AQPIAQD4AQGYAgigAsIJwgIKEAAYRxjWBBiwA8ICBBAhGArCAggQABiJBRiiBJgDAOIDBRIBMSBAiAYBkAYIkgc
```


The API responds with a JSON object containing three fields: short_url (the complete short link ending in /pbkYV6), short_code (just the 6-character code pbkYV6), and long_url (the original Google search URL that was passed in). The shortcode pbkYV6 is now stored in DynamoDB linked to the original URL.

The shortcode pbkYV6 is randomly generated using Python's secrets module to ensure it is URL-safe and unique. Before saving, the Lambda function checks DynamoDB to confirm the code does not already exist. The short_url is constructed as {DOMAIN}/{short_code} using the API Gateway invoke URL as the domain.

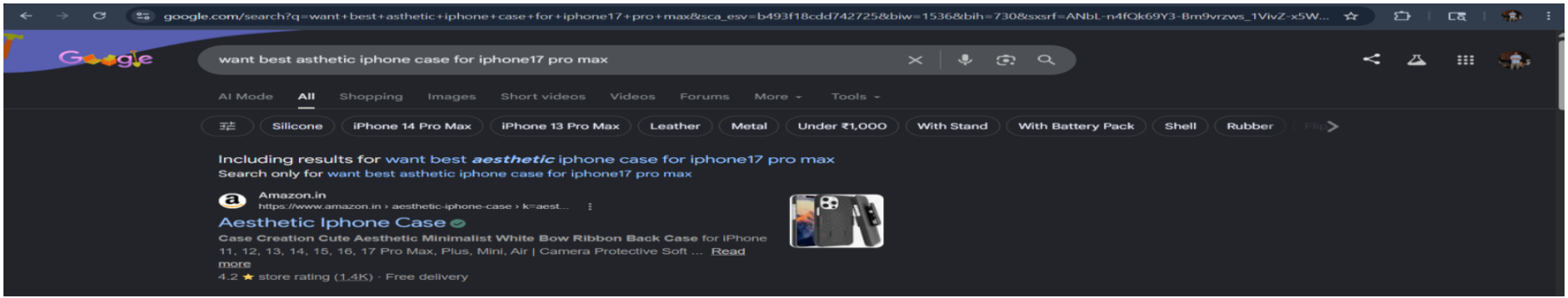
Step 38 — Paste Short URL into Browser



The generated short URL https://qllx65ml7b.execute-api.ap-south-1.amazonaws.com/prod/pbkYV6 is typed into the Chrome address bar. The browser autocomplete already shows it is associated with the original Google search page. Pressing Enter will trigger a GET request to the API Gateway redirect endpoint.

When the browser visits this short URL, it sends a GET request to API Gateway which routes it to MY-LAMBDA-REDIRECT-URL. The Lambda function extracts 'pbkYV6' from the path, calls DynamoDB get_item() to find the original URL, then returns a 301 redirect response with the Location header set to the long Google URL.

Step 39 — Redirect Successful: Original Page Loads



The browser successfully redirected from the short URL to the original Google search results page for 'want best aesthetic iPhone case for iPhone 17 pro max'. The full original Google URL with all parameters is now shown in the browser address bar. The serverless URL shortener is fully working end-to-end.

This final screenshot confirms the complete working flow: POST request creates a short URL → DynamoDB stores the mapping → GET request to the short URL → Lambda retrieves the mapping → 301 redirect response → browser loads the original page. The entire serverless URL shortener built with AWS Lambda, DynamoDB, API Gateway, Docker, ECR, and Terraform is live and functional.